CSC3050 Project 4: Cache Simulator Report

Group Members: 金家驹 & 袁新宇

Student ID: 123090236 & 123090778

Contribution: 50% & 50%

Note: Only leave sections blank if they are not yet implemented. We will verify that your code matches the data in your report. Any inconsistencies will result in double point deductions.

1. Part 1

1.1 Split Cache Design

Description of how the split cache (ICache and DCache) is implemented and the main differences compared to unified cache.

We implemented the split cache through the method below:

1. Derive ICache from the Cache class and override the setByte() function to ignore writes, since instruction cache is read-only in this project.
2. Process I-type and D-type memory access differently by only execute read for ICache.

The main difference is that split cache supports parallel access to instructions and data, improving throughput and reducing contention compared to a unified cache where all memory accesses are serialized through a single cache.

1.2 Implementation Details

Detailed explanation of key implementation methods, especially the design of the ICache class and the simulation method for parallel access.

1.2.1 ICache Inheritance

The ICache inherits from Cache and overrides the setByte() method to print a warning message and skip the write, which mimics the behavior of instruction caches that are often read-only.

1.2.2 Memory Dispatch Logic

In the simulation loop, each memory operation is dispatched based on its type. If the type is I, only use getByte(); if the type is D, use both getByte() and setByte(). This ensures that instructions and data are handled independently.

1.2.3 Statistics Implementation

The two types of cache each maintains its own Cache::Statistics structure, tracking reads, writes, hits, misses and total cycles. By adding two total cycles together, we can get the correct result.

1.3 Result Analysis

Trace File	Cache Type	Miss Count	Expected Miss Count	Total Cycles	Expected Total Cycles
I.trace	Split Cache	512	512	41088	41088
I.trace	Unified Cache	384	384	54912	54912
D.trace	Split Cache	634	634	67584	67584
D.trace	Unified Cache	507	507	68141	68141

Analysis of performance differences between split cache and unified cache for different trace files, explaining the observed results.

1.3.1 I.trace

For I.trace, which is instruction-heavy, the split cache performs better. Even though it has more misses, it avoids interference from data access, leading to lower total cycles.

1.3.2 D.trace

For D.trace, which is data-heavy, the unified cache has fewer misses due to its larger shared space. However, its total cycles are only slightly better or comparable to the split version.

1.3.3 Conclusion

According to the results above, we should: use split cache for instruction-dominant workloads; use unified cache for data-heavy workloads. Furthermore, performance depends on the access pattern and how well the cache layout matches it

2. Part 2

Correct results after bug fixed:

Cache Level	Read Count	Write Count	Hit Count	Miss Count	Miss Rate	Total Cycles
L1	181,708	50,903	177,911	54,700	23.5%	717,519
L2	47,667	7,033	25,574	29,126	53.1%	888,292
L3	26,984	2,142	21,596	7,530	25.9%	1,184,920

My Results after bug fixed:

Cache Level	Read Count	Write Count	Hit Count	Miss Count	Miss Rate	Total Cycles
L1	181708	50903	177911	54700	23.516	717519
L2	54700	12751	25608	29092	53.185	886984
L3	29092	5014	21562	7530	25.883	1184240

3. Part 3

Optimization Techniques Application and Results

(Choose one optimization technique for each trace file)

No Optimization means, just use the initial code to run simulation.

Trace	Optimization	Miss Count	Miss Rate	Expected Miss Rate
test1.trace	No Optimization	102656	99.257	None
test1.trace	FIFO	384	0.371	0.371%
test2.trace	No Optimization	100013	98.532	None
test2.trace	Prefetching	3429	3.378	5.803%
test3.trace	No Optimization	51712	50.124	None
test3.trace	Victim Cache	384	0.372	0.372%

4. Part 4

4.1 Performance Comparison (L1 Level)

Implementation	Read Count	Write Count	Hit Count	Miss Count	Miss Rate	Total Cycles
matmul0	786432	262144	716581	331995	31.662	3665589
matmul1	528384	4096	258741	273739	51.408	2480901
matmul2	790528	266240	1001365	55403	5.243	1767413
matmul3	790528	266240	493861	562907	53.267	7101725
matmul4	540672	16384	526103	30953	5.557	859279

4.2 Analysis of Performance Differences between matmul1, 2, 3

1. matmul1

In matmul1 (i-j-k order), the A matrix is accessed row-wise with good spatial locality, but the B matrix is accessed column-wise, leading to poor cache usage. Every access to B[k][j] results in large memory jumps, causing frequent cache misses and inefficient use of cache lines. Although C[i][j] is temporarily stored in a register to minimize memory writes, the poor locality of B dominates, resulting in a very high L1 cache miss rate of $O(n^3)$ and making matmul1 relatively inefficient for large matrices.

2. matmul2

In matmul2 (k-i-j order), the loops are reordered to improve cache behavior significantly. Here, B[k][j] is accessed continuously across memory, providing excellent spatial locality, and C[i][j] is updated row-by-row efficiently. Although A[i][k] is less ideal spatially, storing it in a register (aik) ensures high temporal reuse. This design minimizes unnecessary memory traffic and cache misses, achieving an overall L1 miss rate of only $O(n^2)$, making matmul2 the best choice for performance among the three methods.

3. matmul3

In matmul3 (j-k-i order), B’ s access is partially improveed by caching B[k][j] into a register (bkj), but suffers heavily from inefficient access to C[i][j]. As the innermost loop iterates over i, C[i][j] updates involve large memory jumps, disrupting cache line efficiency. Despite the optimization for B, the poor memory access pattern for C results in frequent cache misses and overall performance degradation similar to matmul1, with an L1 miss rate of about $O(n^3)$.